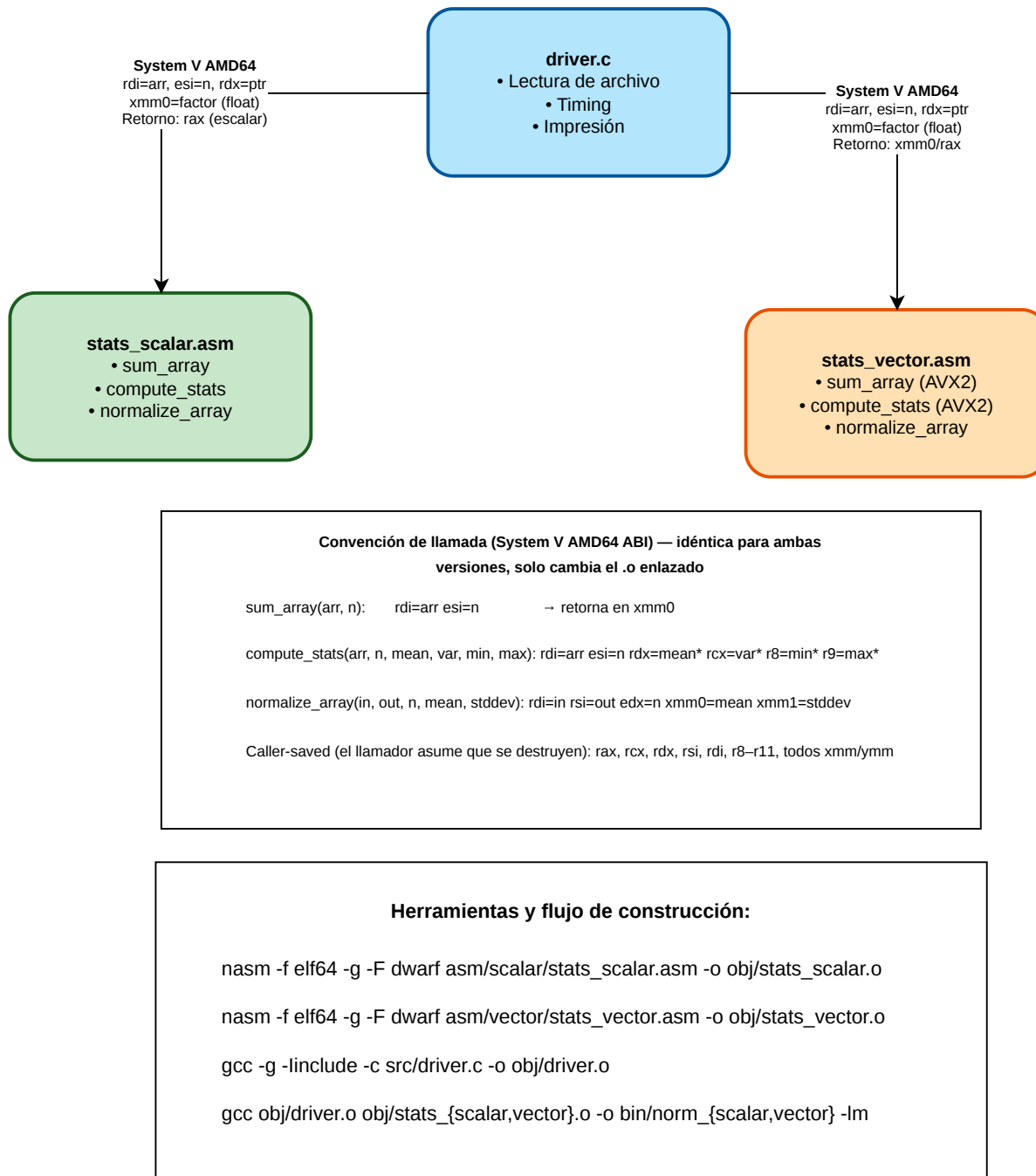
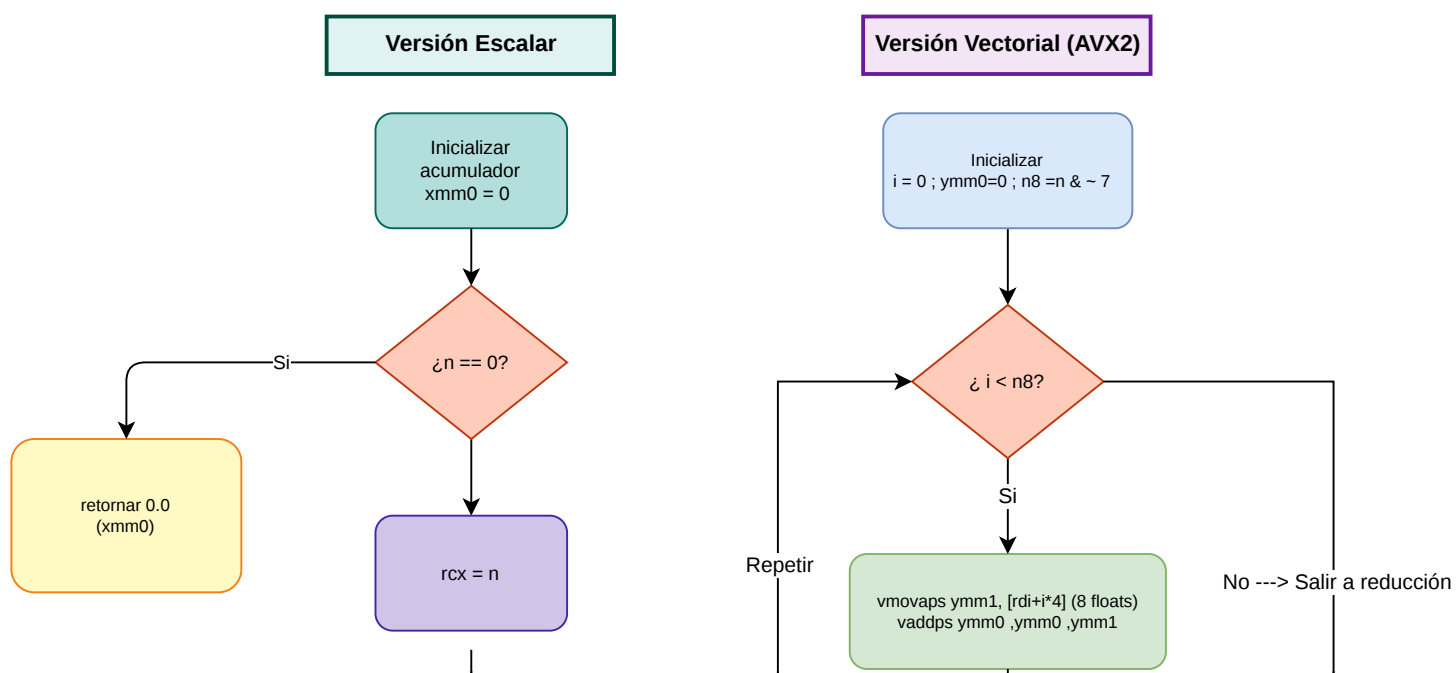
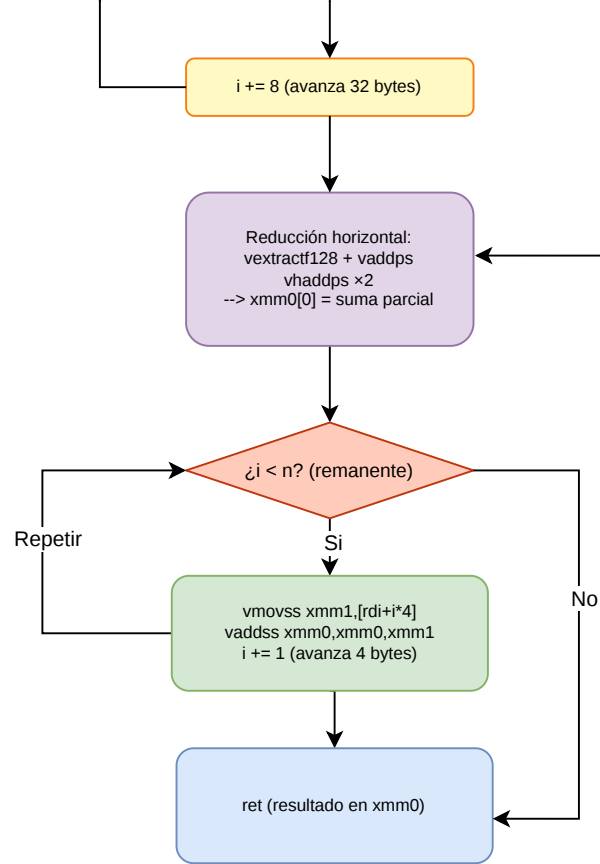
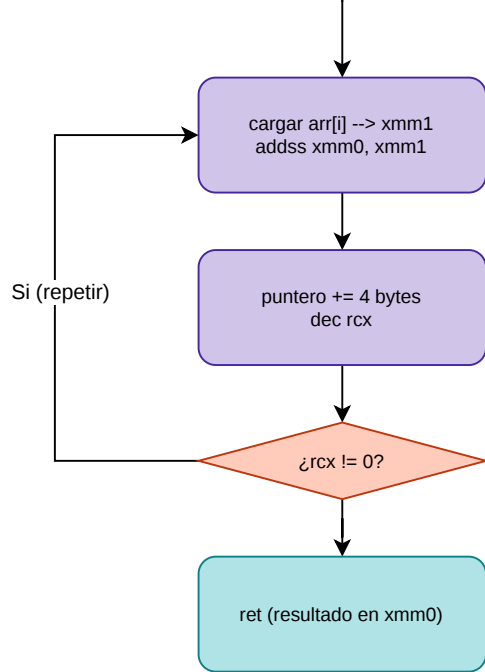


1. Diagrama de Bloques: Arquitectura del Proyecto



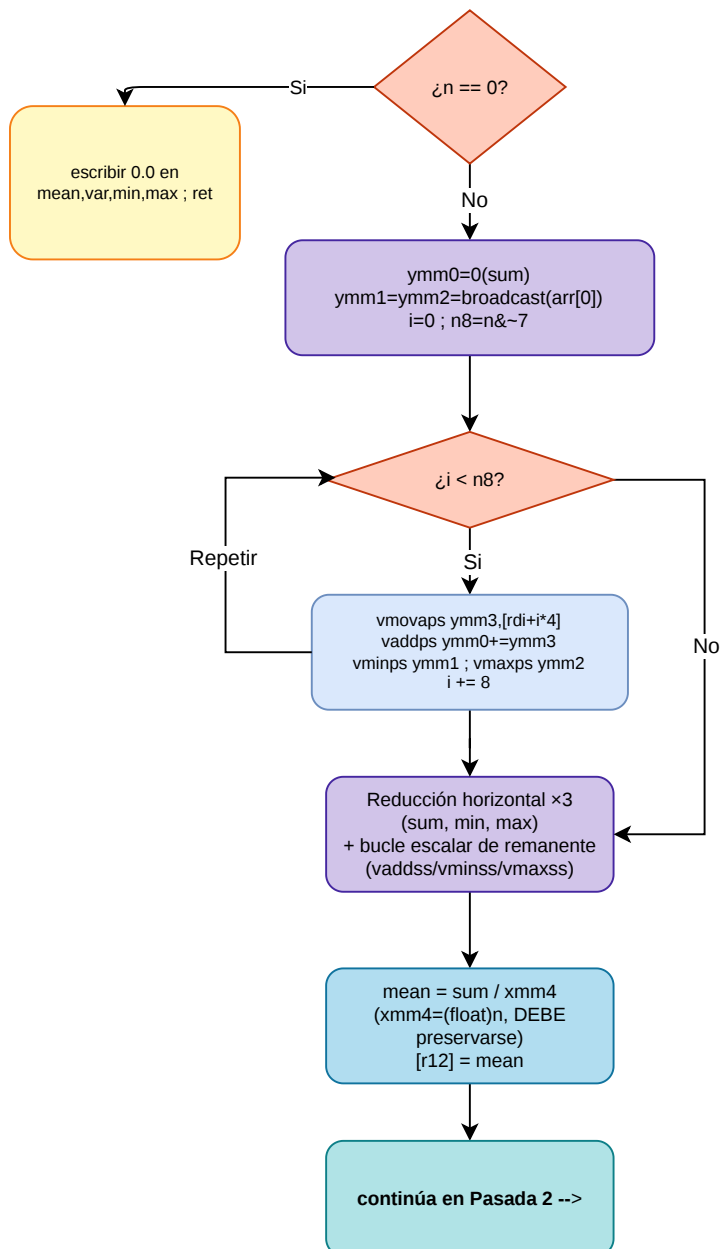
2. Diagrama de Flujo de Control: sum_array



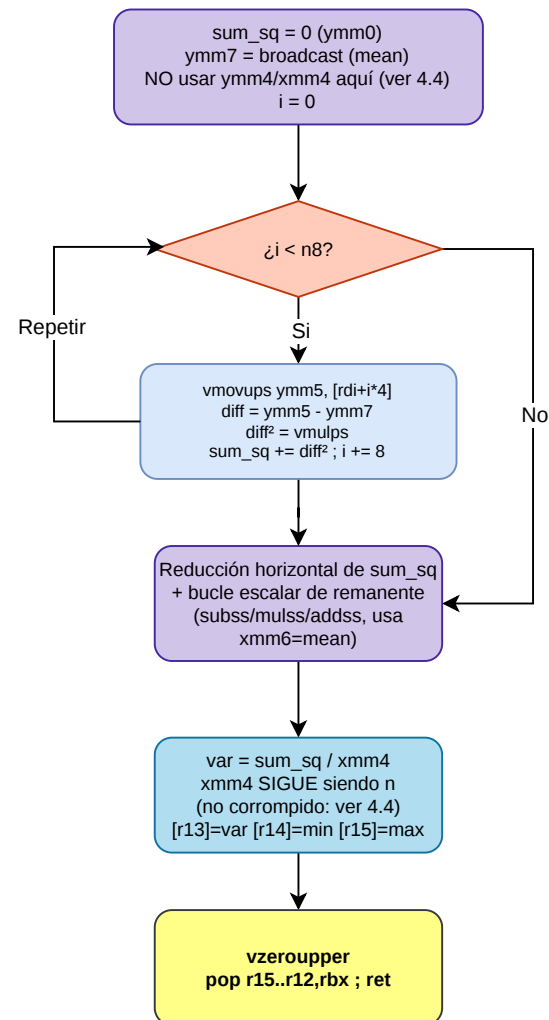


3. Flujo de control compute_stats (vectorial), dos pasadas

Pasada 1: suma, mínimo, máximo



Pasada 2: varianza (usa mean de la pasada 1)



4. Tabla de asignación de registros -- compute_stats (vectorial)

Registro	Propósito
rdi	arr (puntero, argumento 1)
esi	n (argumento 2)
rdx ---> r12	mean* (guardado en callee-saved)
rcx ---> r13	var* (guardado en callee-saved)
r8 ---> r14	min* (guardado en callee-saved)
r9 ---> r15	max* (guardado en callee-saved)
eax	índice i del bucle
ecx / r10d	n8 = n & ~7 (límite del bucle vectorial)
ymm0 / xmm0	acumulador de suma sum_sq (pasada 2)
ymm1 / xmm1	acumulador de mínimo
ymm2 / xmm2	acumulador de máximo
ymm3 / xmm3	temporal: bloque de 8 floats cargado de arr
xmm4	(float)n debe preservarse hasta la división final
xmm6	media (mean), preservada durante toda la pasada 2
ymm7	broadcast de la media (NO usar ymm4 aquí: ver 4.4)
ymm5 / xmm5	temporal de la pasada 2: diff = arr - mean

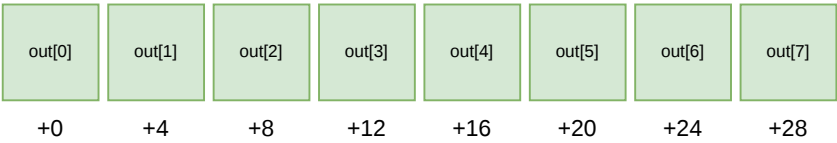
5. Mapa de memoria -- arreglos de entrada (in) y salida (out)

Ambos arreglos reservados con aligned_alloc(32, ...) en driver.c (bases múltiplo de 32)

Arreglo IN (entrada) -- base múltiplo de 32

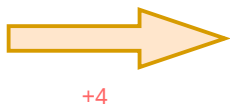


Arreglo OUT (salida) mismo N y base también múltiplo de 32



out[i] = (in[i] - mean) / stddev
normalize_array recorre IN y OUT con el MISMO índice i. Como ambas bases son múltiplo de 32 y el offset (i*4) es idéntico en los dos arreglos, las cargas de IN y los almacenamientos en OUT del bucle vectorial (vmovaps) permanecen alineados a la vez.

Avance ESCALAR (rdi += 4 / rsi(out) += 4 cada iteración):



Avance VECTORIAL (índice $i \pm 8$ desplazamiento $i*4 \pm 32$, en ambos arreglos):



+32 bytes (un solo salto, de elem[0] a elem[8])

- vmovaps exige dirección múltiplo de 32 (falla con #GP si no); vmovups no lo exige.
- Como ambas bases (in y out) son múltiplo de 32 y el índice avanza de 8 en 8 floats (32 B), TODAS las direcciones de carga Y almacenamiento del bucle vectorial permanecen alineadas.
- El remanente ($n \% 8$) se procesa con vmovss/movss, 1 float (4 bytes) a la vez en ambos arreglos, sin requisito de alineación de 32 bytes.